EasySave - Technical Architecture

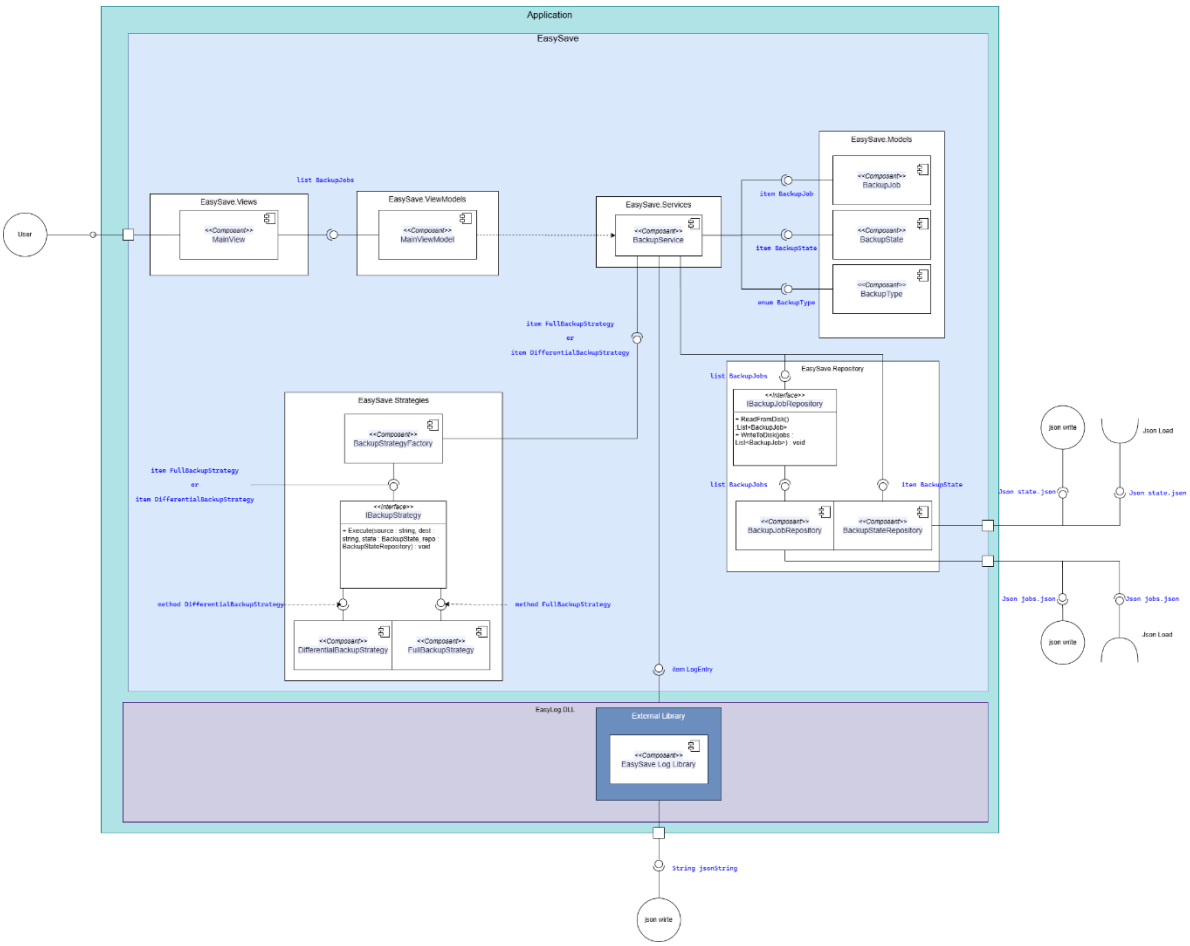
This document details the internal structure of the **EasySave** application. It explains the role of each component and the design patterns selected to ensure a modular and maintainable code base.

1. Global Architecture Overview

The application follows a multi-layered architecture based on the MVVM pattern, separating the User Interface, Business Logic, and Data Access.

The 4 Main Layers:

1. Presentation Layer (Views): A modern WPF interface (XAML) replacing the old console display.
2. Application Layer (ViewModels): Binds the View to the Model.
3. Business Logic Layer (Services & Strategies): The "Brain" of the application.
4. Data Access Layer (Repositories): Reads/Writes JSON files.



2. Component Detail

A. The Business Logic (Service Layer)

This is the Orchestrator. It contains the main business logic but does not perform low-level tasks (like copying files or writing JSON) itself.

Class: BackupService	Responsibilities: Validates rules (e.g., max 5 jobs). Calculates new IDs. Calls the Factory to get the right algorithm. Delegates execution to the Strategy.
Class : ProcessChecker	Checks if a specific "Business Software" is running before starting a job to prevent file conflicts.

B. The Algorithms (Strategy Pattern)

To manage different backup types efficiently, the Strategy Pattern was implemented. Instead of using complex conditional statements (like if/else) inside the service to distinguish between "Full" and "Differential" backups, we defined a common contract interface named **IBackupStrategy**.

This interface is implemented by two concrete classes: **FullBackupStrategy**, which copies all files recursively, and **DifferentialBackupStrategy**, which performs a smart scan to copy only files that have been modified or created since the last backup. This approach allows developers to add new backup types in the future (e.g., Incremental) without modifying the existing Service code.

C. Object Creation (Factory Pattern)

To further decouple the Service from the specific strategy implementations, we utilized the Factory Pattern. The **BackupStrategyFactory** class is responsible for the

instantiation logic. When the Service needs to execute a job, it simply provides the job type to the Factory, which returns the appropriate strategy instance. This ensures that the Service remains unaware of the concrete classes it is using, adhering to the Dependency Inversion Principle.

D. Data Persistence (Repository Pattern)

Direct interaction with the file system for data storage is abstracted via the Repository Pattern. This layer isolates the application from the specific details of data storage, meaning that switching from JSON files to a SQL database would only require changes in this specific layer.

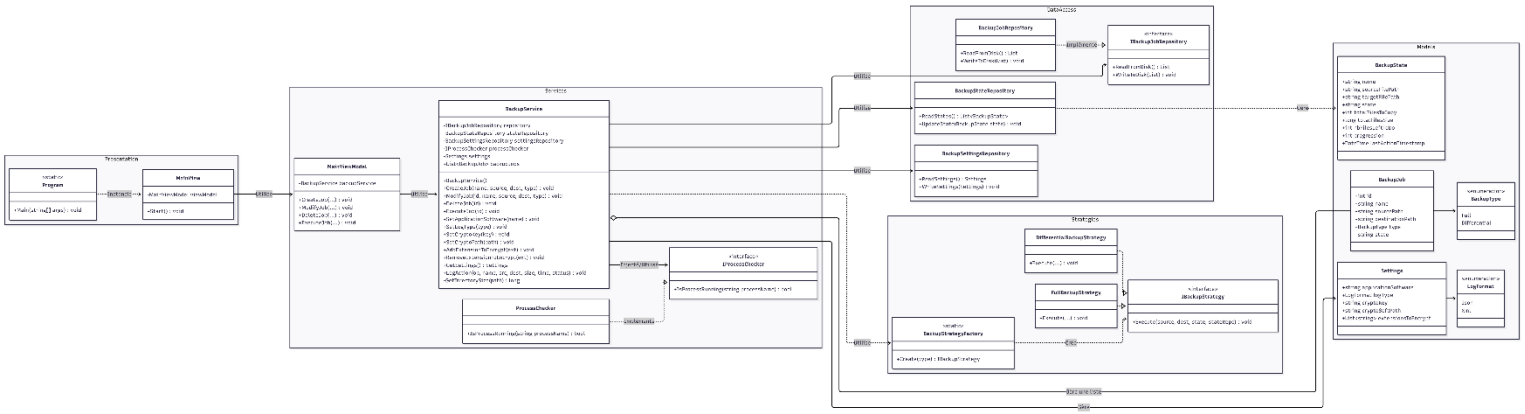
The **BackupJobRepository** handles the persistence of configuration data. It manages the reading and writing of the jobs.json file, ensuring that the user's backup configurations are saved and retrieved correctly.

The **BackupStateRepository** plays a more critical role. It manages the state.json file, which provides real-time monitoring of the backup progress. Since the backup strategies write to this file continuously—potentially hundreds of times per second—we implemented a Thread-Safe Locking mechanism. A static lock object ensures that only one process can write to the state file at any given moment, preventing file corruption and access conflicts during high-speed execution.

E. Settings & Security Management

A new configuration system has been implemented to handle application-wide settings.

- Settings Model: Stores configuration such as:
    - applicationSoftware: The name of the business software to monitor (e.g., *Calculator.exe*).
    - cryptoKey: The encryption key used for secure backups.
    - extensionsToEncrypt: A list of file extensions (e.g., .txt, .docx) that must be encrypted.
    - logType: Preferred logging format (json or xml).
- BackupSettingsRepository: Persists these settings in a dedicated settings.json file.



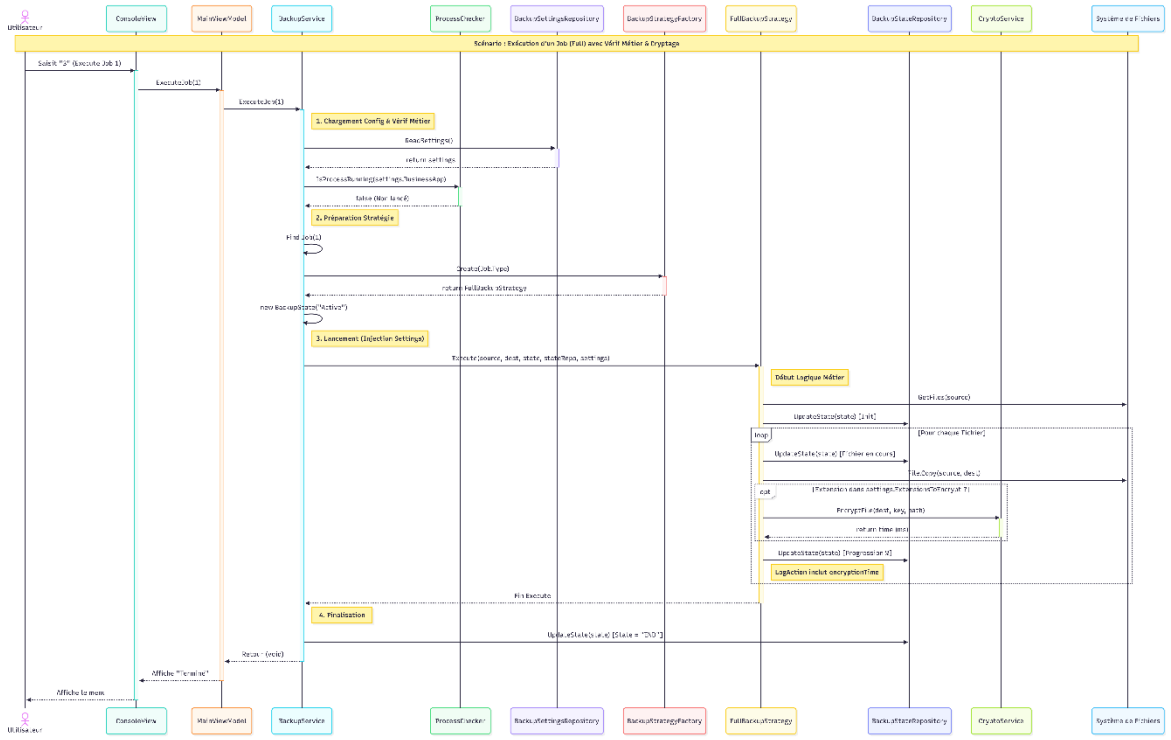
DiagrammeClasse.png

3. Data Flow Execution (Step-by-Step)

When a user executes a job (e.g., "Execute Job 1"), the following chain reaction occurs:

1. View: The user clicks "Run". The View sends the command to the ViewModel.
2. ViewModel: Calls service.ExecuteJob(1).
3. Service:
    - Retrieves the Job info (Source/Destination).
    - Calls BackupStrategyFactory.Create(job.Type).
    - Receives a strategy (e.g., DifferentialBackupStrategy).
    - Calls strategy.Execute(...).
4. Strategy:
    - Scans files using System.IO.
    - Loop: For each file:
        - Updates the State object.
        - Calls stateRepository.UpdateState() (Writes to JSON).
        - Copies the file.
5. Repository: Writes the JSON to the disk safely.

Exemple Sequence Diagram:



EasySave-Sequence-FullV2.png

4. Error Management and Resilience

A critical requirement for backup software is stability. The application implements a robust error handling strategy to ensure that the backup process does not terminate unexpectedly, even when encountering file system issues.

Specifically, the file copy logic is wrapped in strict try-catch blocks. Common scenarios, such as a file being locked by another process (e.g., an open Word document) or access rights restrictions, are caught as IOException or UnauthorizedAccessException. Instead of crashing the entire application, the system logs the specific error to the console, skips the problematic file, and proceeds to the next one. This ensures that the backup job reaches completion to the maximum extent possible, regardless of individual file errors.

5. Command Line Interface (CLI) and Automation

To support integration with external scheduling tools (like Windows Task Scheduler) or scripts, EasySave includes a dedicated Command Line Interface (CLI). This logic is

handled directly in the **Program**.cs entry point, bypassing the graphical user interface entirely when arguments are detected.

The application features a custom argument parser capable of interpreting complex patterns. It supports ranges (e.g., 1-3 to run jobs 1 through 3) and specific lists (e.g., 1;3 to run only jobs 1 and 3). This architecture allows the software to be used in "headless" mode for automated nightly backups without requiring user interaction.

6. Data Structure and Storage

All application data is persisted locally using JSON serialization, ensuring readability and ease of debugging. The data is split into two distinct files located in the application's root directory:

- Configuration (jobs.json): This file stores the static definition of the backup jobs. For each job, it persists the unique ID, the friendly name, the source directory path, the destination directory path, and the selected backup strategy (Full or Differential).

  Example :

```
[
  {
    "id": 1,
    "name": "Save_Documents",
    "sourcePath": "C:\\Users\\Admin\\Desktop\\Source",
    "destinationPath": "C:\\Users\\Admin\\Documents\\Backup",
    "type": 0,         // Enum: 0 = Full, 1 = Differential
    "state": "Inactive"
  }
]
```

- Monitoring (state.json): This file acts as a real-time dashboard. It is updated dynamically during execution to reflect the current progress. Key data fields include the total number of files to copy, the file currently being processed, the progression percentage, and the current status (Active or End).

Example :


```
[
  {
    "name": "Save_Documents",
    "sourceFilePath": "C:\\Source\\image.png",
    "targetFilePath": "C:\\Backup\\image.png",
    "state": "ACTIVE",
    "totalFilesToCopy": 1480,
    "totalFilesSize": 263149754,
    "nbFilesLeftToDo": 571,
    "progression": 61,
    "lastActionTimestamp": "2026-02-03T11:20:18.518"
  }
]
```


Global Configuration (settings.json): This file stores application-wide settings that apply to the general behavior of the application and security features. It persists the Business Software to monitor (to prevent conflicts during execution), the Encryption parameters (key, software path, and specific file extensions to encrypt), and the preferred Log Format (JSON or XML).

Example :


```
{
  "cryptoSoftPath": "C:\\Users\\bapti\\Desktop\\CryptoSoft\\bin\\Debug\\net8.0\\win-x64\\CryptoSoft.exe",
  "cryptoKey": "test",
  "extensionsToEncrypt": [
    ".txt"
  ],
  "logType": "Json",
  "applicationSoftware": "notepad.exe"
}
```


7. Technical Environment

The project is built on modern .NET technologies to ensure cross-platform compatibility and performance.

- Framework: .NET 8.0 (Long Term Support) / .NET 10.0 (Preview).
- UI Framework: WPF (Windows Presentation Foundation) (Replacing Console).
- Operating System: Primary development and testing were conducted on Windows 10/11 due to specific file system interactions, but the Core architecture is compatible with Linux and macOS.
- IDE: Developed using Visual Studio 2022